

Lab3 理论测试

* 您的姓名：

* 学号：

* 1. 当前队列是由 TAILQ 实现的队列，首指针为 head，尾指针为 tail，下列选项中正确的为（）

- ☐ A. head -> tqh_last = &tail
- ☐ B. head -> tqh_last = &tail->field.tqe_next
- ☐ C. head -> tqh_last = tail->field.tqe_next
- ☐ D. tail = *(*(head->tqh_last)->tqh_last)

* 2. 在 map_segment 函数中，通过循环调用 page_insert 完成连续虚拟地址段到物理地址段的映射，而不是直接使用 pgdir_walk 获取页表项并手动赋值。以下关于使用 page_insert 的原因，不正确的一项是？

- ☐ A. page_insert 会自动检测目标虚拟地址是否已有映射，若存在且映射到不同物理页，则先解除旧映射并释放相应资源
- ☐ B. page_insert 会自动增加物理页的 pp_ref 引用计数，确保映射期间该页不会被回收

- ☐ C. page_insert 会在修改页表项后调用 tlb_invalidate 刷新 TLB，避免 CPU 使用过时的地址转换缓存
- ☐ D. page_insert 会自动为尚未分配的物理页调用 page_alloc 分配新页框，简化映射流程

* 3. 下列有关 ASID 的说法中，正确的为 ()

- ☐ A. 如果每次切换页表时无效化 TLB 表项，即使不使用 ASID 操作系统也可以运行。
- ☐ B. MOS 系统已使用 256 个 ASID，仍能正常创建下一个进程。
- ☐ C. ASID 用于标识虚拟地址空间，降低 TLB 命中次数，有效提高处理器性能。
- ☐ D. TLB 实质创建了一个 $\langle \text{VPN}, \text{ASID} \rangle \rightarrow \langle \text{PFN}, \text{C}, \text{D}, \text{V}, \text{G} \rangle$ 的映射。每个进程都有自己独立的物理地址空间下的一套 TLB 缓存。

* 4. 通过页目录自映射，用户进程可以在用户态只读地访问自己的页表。虚拟地址 UVPT 处被映射为页目录本身的副本。请完成以下函数，该函数给定一个虚拟地址 va，返回该地址对应的页表项的虚拟地址: `u_long get_pde_va(u_long va) { return UVPT + (_____ << 12) + (_____ * sizeof(Pte)); }`

- ☐ A. PDX(va), PTX(va)
- ☐ B. PDX(va), PDX(va)
- ☐ C. PTX(va), PDX(va)
- ☐ D. PTX(va), PTX(va)

* 5. 在 env_alloc 中，env_tf.cp0_status 被设置为 STATUS_IM7 | STATUS_IE | STATUS_EXL | STATUS_UM。当进程第一次被调度时，RESTORE_ALL 宏将该值写入 CPU 的 Status 寄存器，随后执行 eret 指令。下列关于标志位的说法，错误的一项是 ()

- ☐ A. 在 eret 执行前，由于 EXL=1，即使 IE=1 且 IM7=1，时钟中断也无法被响应，必须等到 eret 清零 EXL 后才会响应。
- ☐ B. eret 指令会原子性地完成:将 EPC 写入 PC，并将 Status 寄存器的 EXL 位清零，从而退出异常模式并返回用户态。
- ☐ C. 当且仅当 EXL 被设置为 1 且 UM 被设置为 0 时，处理器处于内核模式。

- ☐ D. 若在 env_alloc 中忘记设置 EXL，则 RESTORE_ALL 后 CPU 立即进入用户模式，此时 PC 仍指向内核中的 eret 指令地址，将触发地址错误异常。

* 6. 将可执行文件加载至指定的进程地址空间的函数是 ()

- ☐ A. load_icode
- ☐ B. load_icode_mapper
- ☐ C. elf_load_seg
- ☐ D. env_create

* 7. 当 MIPS CPU 发生地址错误 (Address Error) 或 TLB 缺失 (TLB Miss) 时，硬件会自动将引发错误的虚拟地址记录在哪个 CP0 寄存器中？

- ☐ A. Status 寄存器
- ☐ B. Cause 寄存器
- ☐ C. EPC 寄存器
- ☐ D. BadVAddr 寄存器

* 8. 在 MIPS 体系结构中，当发生 TLB Miss 异常时，关于 EntryHi 寄存器内容的填充，下列说法正确的是？

- ☐ A. 操作系统在异常处理程序中同时填入 VPN 和 ASID
- ☐ B. 硬件自动提取引发错误的 VPN 填入高位，ASID 则由软件在进程切换时提前填入低位
- ☐ C. 硬件自动同时填入引发错误的 VPN 和当前进程的 ASID
- ☐ D. 操作系统在进程切换时同时提前填入 VPN 和 ASID

* 9. 当 MIPS CPU 捕获到一个异常（错误或中断）时，硬件会自动修改 Status 寄存器中的 EXL (Exception Level) 位。这一动作的目的是什么？

- ☐ A. 将 EXL 置为 0，使 CPU 进入用户态运行异常处理程序
- ☐ B. 将 EXL 置为 1，强制 CPU 进入内核态并全局禁止中断，防止异常处理现场

被破坏

- ☐ C. 将 EXL 置为 1, 清空 TLB 以防止后续发生地址翻译错误
- ☐ D. 将 EXL 置为 0, 开启嵌套中断允许更高优先级的中断抢占

* 10. 操作系统在执行异常分发程序 (如 `exc_gen_entry`) 时, 如何判断引发本次异常的具体原因 (例如是系统调用、只读页写入错误还是时钟中断) ?

- ☐ A. 读取 `BadVAddr` 寄存器的值并进行地址对齐计算
- ☐ B. 读取 `Status` 寄存器中的 `IM` (Interrupt Mask) 位域
- ☐ C. 读取 `Cause` 寄存器中的 `ExcCode` (异常码) 位域, 以此作为索引查找异常向量表
- ☐ D. 读取 `EPC` 寄存器指向的指令, 通过软件反汇编解码错误类型

* 11. 在异常现场保存 (`SAVE_ALL`) 阶段, 操作系统需要判断当前异常是否是在内核态触发的 (即"异常重入"或内核态代码执行错误), MIPS 系统在软件中是通过检查什么来判断的?

- ☐ A. `Cause` 寄存器中的硬件中断引脚状态
- ☐ B. `Status` 寄存器中的 `UM` (User Mode) 位是否为 0
- ☐ C. 触发异常的 PC 地址是否位于 `0x80000000` 之上
- ☐ D. `EntryHi` 寄存器中的 `ASID` 是否为 0

* 12. MIPS 中针对不同的异常类型设计了不同的硬件入口地址。当发生普通的内存访问错误 (如向只读页写入) 或硬件中断时, CPU 默认跳转到哪个物理地址执行错误分发程序?

- ☐ A. `0x00000000`
- ☐ B. `0x80000000`
- ☐ C. `0x80000180`
- ☐ D. `0xBF000000`

* 13. 当软件成功处理完一个错误或异常，准备将控制权交还给被中断的用户程序时，需要执行 MIPS 特权指令 `eret`。关于 `eret` 指令的行为，以下描述最准确的是？

- ☐ A. 跳转到 EPC 寄存器指向的地址，并原子性地将 Status 寄存器的 EXL 位清零以恢复用户态和中断响应
- ☐ B. 仅恢复通用寄存器的状态，跳转地址需要由软件通过 `jr ra` 指令单独完成
- ☐ C. 重新加载页表基址寄存器，并刷新整个 TLB 缓存
- ☐ D. 跳转到 BadVAddr 寄存器指向的地址重新执行导致错误的指令

* 14. 在 `elf_load_seg` 函数中，需要处理多种页面加载情况。以下说法正确的有（ ）？【多选题】

- ☐ A. 起始页可能因 `va` 未对齐而有偏移，该页可能只含数据，或同时含数据和需清零区域。
- ☐ B. 中间页若完全在 `[va, va+bin_size)` 内则拷贝数据；若完全在 `[va+bin_size, va+sgsize)` 内则清零。
- ☐ C. 末尾页可能部分数据部分清零，也可能完全清零。
- ☐ D. 每次调用 `map_page` 需传递页对齐起始地址、页内偏移、权限、源数据指针及长度。

* 15. 在 `env_run` 中保存当前进程上下文的代码为：`curenv->env_tf = *((struct Trapframe *)KSTACKTOP - 1)`；这里 `((struct Trapframe *)KSTACKTOP - 1)` 指向的地址等价于 `KSTACKTOP -` （用 `sizeof` 表达式填写）。

提交